UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



DESARROLLO DE APLICACIONES I

SISTEMA WEB DE GESTIÓN DE PRODUCTOS – COMPU TECH

INTEGRANTES:

- QUISPE TICA Ruth Nayeli

DOCENTE:

- Raúl Fernández Bejarano

CICLO: IV
SECCIÓN: A1

FECHA: 10 de noviembre del 2026

HUANCAYO - PERÚ
2026

1. Introducción

El presente informe explica el desarrollo y funcionamiento del Sistema Web de Gestión de Productos denominado CompuTech. El proyecto está orientado a organizar el catálogo y el inventario de una tienda de equipos informáticos y accesorios, permitiendo registrar, consultar, buscar, actualizar y eliminar productos mediante una interfaz web. Asimismo, incorpora acceso por usuario, separación de permisos según el rol y reportes básicos para conocer la situación del inventario.

La solución se desarrolla con Java, páginas JSP, Servlets, Maven, CSS y JavaScript, y aplica el patrón Modelo–Vista–Controlador (MVC). De acuerdo con la actividad académica, los productos y usuarios se mantienen temporalmente en colecciones Java en memoria, sin conexión a una base de datos. Esta decisión permite concentrarse en la organización del código, el flujo de las solicitudes web y la lógica de negocio, aunque también implica que los datos no son permanentes.

El análisis se basa en los archivos del proyecto comprimido y en los requisitos descritos en la Actividad N.° 04 de Desarrollo de Aplicaciones I. Se explican los componentes identificables en el código y se distinguen las características implementadas de las mejoras que podrían incorporarse en una versión futura.

2. Objetivos

2.1 Objetivo general

Explicar de manera ordenada la estructura, las tecnologías, la arquitectura y las funcionalidades del Sistema Web de Gestión de Productos CompuTech, destacando cómo se aplica el patrón MVC en una aplicación web Java sin base de datos.

2.2 Objetivos específicos

- Identificar la función de las tecnologías Java, JSP, Servlets, Maven, CSS y JavaScript dentro del proyecto.
- Describir la responsabilidad del modelo, las vistas, los controladores y los servicios.
- Explicar los módulos de inicio, productos, reportes, información, registro e inicio de sesión.
- Reconocer las validaciones de datos y las restricciones de acceso por rol.
- Precisar las limitaciones del almacenamiento en memoria y plantear recomendaciones de mejora.

3. Descripción general del sistema

CompuTech es una aplicación web para administrar productos relacionados con computadoras, laptops, monitores, periféricos, componentes, almacenamiento, redes y audio. Cada producto contiene datos como código, nombre, marca, categoría, precio, stock, estado, descripción e imagen. Estos atributos permiten identificar el artículo, mostrar sus características y calcular su valor dentro del inventario.

El sistema maneja dos perfiles principales. El administrador puede registrar productos nuevos, listar el inventario, buscar artículos, editar sus datos y eliminarlos con confirmación. El usuario registrado puede revisar el catálogo, consultar detalles, buscar productos y acceder al reporte de artículos disponibles. Esta separación evita que las operaciones de modificación estén disponibles desde la interfaz ordinaria del cliente.

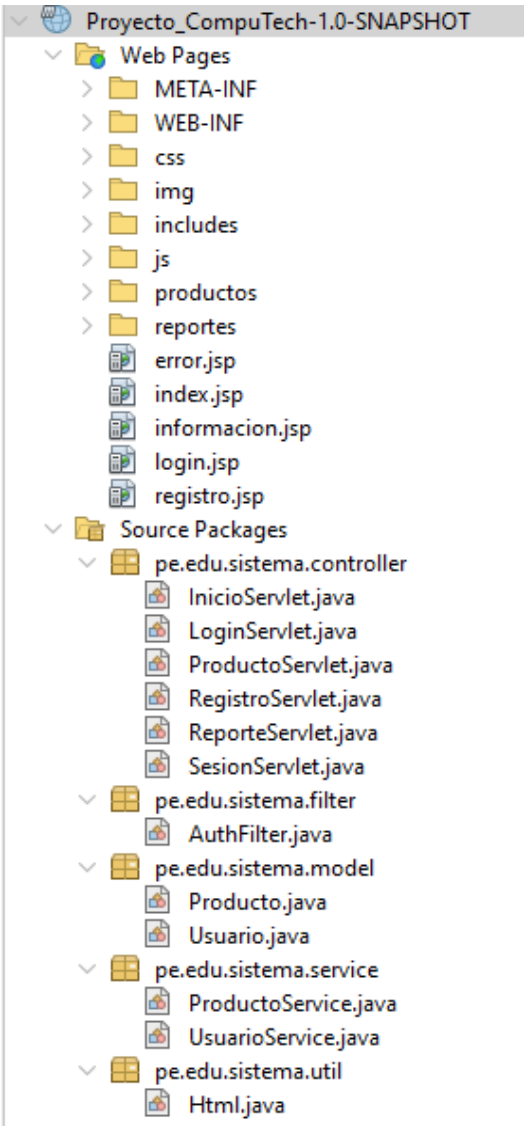
La aplicación incluye productos de ejemplo cargados al iniciar el servicio. El catálogo y los reportes utilizan esos registros para presentar información demostrativa. El sistema no procesa compras ni pagos en el código revisado; su propósito principal es la gestión y consulta de productos.

4. Tecnologías utilizadas

Tecnología	Función en el proyecto
Java	Lenguaje principal de la lógica de negocio. Define las clases de productos y usuarios, administra las colecciones y ejecuta las reglas de validación y consulta.
JSP (JavaServer Pages)	Construye las páginas que se envían al navegador. Las vistas muestran datos que los controladores colocan en la solicitud y usan etiquetas JSTL para recorrer listas y presentar condiciones.
Servlets	Reciben solicitudes HTTP, identifican la acción solicitada, llaman a los servicios, preparan datos para las vistas y deciden si se debe mostrar una página o redirigir al usuario.
Maven	Organiza el proyecto, declara dependencias en pom.xml y facilita la compilación y el empaquetado de la aplicación.
CSS3	Define la presentación visual: distribución, colores, formularios, tablas, tarjetas, botones y adaptación de la interfaz.

JavaScript	Apoya las validaciones del lado del navegador y las interacciones de la interfaz, como la confirmación antes de eliminar un producto.
Apache Tomcat 11	Servidor web/servlet previsto para ejecutar la aplicación y procesar las solicitudes de los Servlets.
ArrayList y colecciones Java	Almacenan temporalmente los productos y usuarios en memoria, en lugar de una base de datos.
JDK 17 o superior	Entorno de desarrollo Java indicado en el enunciado de la actividad.

4.1. Estructura



5. Arquitectura Modelo–Vista–Controlador (MVC)

MVC es un patrón de diseño que separa una aplicación en tres responsabilidades relacionadas. El Modelo representa los datos y las reglas de negocio; la Vista se ocupa de presentar la información; y el Controlador recibe las solicitudes y coordina el trabajo entre el modelo y las vistas. Esta separación hace que el código sea más fácil de entender, mantener y ampliar.

5.1 Modelo

La carpeta model contiene las clases Producto.java y Usuario.java. Producto representa un artículo del inventario y encapsula sus atributos mediante campos privados y métodos get/set. También incorpora getValorInventario(), que devuelve el resultado de multiplicar el precio por el stock. Usuario representa las credenciales y los datos de la cuenta, así como el rol que determina los permisos.

5.2 Servicios o lógica de negocio

La carpeta service contiene ProductoService.java y UsuarioService.java. Aunque el servicio no constituye una de las tres letras de MVC, funciona como una capa de apoyo que concentra las reglas de negocio y el acceso a las colecciones. ProductoService mantiene una lista compartida de productos, carga registros iniciales y ofrece operaciones para listar, buscar, registrar, actualizar, eliminar y calcular datos del inventario. UsuarioService administra las cuentas, autentica credenciales y valida los datos del registro.

5.3 Vista

Las vistas están construidas con JSP dentro de src/main/webapp. Entre ellas se encuentran index.jsp, login.jsp, registro.jsp, las páginas de productos, las vistas de reportes e informacion.jsp. Los archivos incluidos header.jsp, menu.jsp y footer.jsp permiten reutilizar partes comunes del diseño y mantener una presentación consistente. Los datos se muestran mediante expresiones JSP y etiquetas JSTL, evitando concentrar toda la lógica de negocio dentro de la página.

5.4 Controlador

Los Servlets de la carpeta controller reciben las solicitudes. ProductoServlet atiende las acciones del módulo de productos; ReporteServlet prepara la información de los reportes; InicioServlet calcula los datos que aparecen en la página principal; LoginServlet procesa el inicio y cierre de sesión; y RegistroServlet gestiona la creación de cuentas. El controlador consulta los servicios y reenvía la solicitud a la vista apropiada.

5.5 Recorrido de una solicitud

- El usuario pulsa una opción o envía un formulario desde una página JSP.
- La solicitud HTTP llega al Servlet correspondiente, por ejemplo /productos.
- El controlador identifica la acción solicitada y comprueba los permisos necesarios.
- El controlador llama a ProductoService o UsuarioService para ejecutar la operación.
- El resultado se incorpora como atributo de la solicitud o de la sesión.
- El Servlet reenvía la solicitud a una página JSP, que presenta el resultado al usuario.

Ejemplo: cuando el administrador registra un producto, el formulario de productos/formulario.jsp envía los campos al endpoint /productos con la acción guardar. ProductoServlet recoge los valores, solicita al servicio que valide y registre el objeto y luego devuelve una respuesta que permite visualizar el listado o el mensaje correspondiente.

6. Explicación de los componentes principales

6.1 Producto.java

Esta clase modela la información de un producto. Sus atributos incluyen código, nombre, marca, categoría, precio, stock, estado, descripción e imagen. El constructor permite crear un objeto con sus datos iniciales, mientras que los métodos get y set permiten consultar o modificar cada atributo. El método getValorInventario() calcula precio × stock, por lo que ayuda a estimar el valor de las unidades registradas.

6.2 Usuario.java

La clase Usuario representa a la persona que accede al sistema. Incluye datos de identificación y un rol, con el que se diferencia al administrador del usuario común. El rol es importante porque la interfaz y los controladores utilizan esa información para decidir qué operaciones están permitidas.

6.3 ProductoService.java

ProductoService centraliza el manejo de productos en una lista ArrayList. El patrón de instancia única utilizado en el servicio permite que los Servlets accedan a la misma colección durante la ejecución de la aplicación. En el constructor se cargan productos de ejemplo, y las categorías y estados se mantienen en listas definidas por el propio servicio. Esta organización evita duplicar la lógica de inventario en cada Servlet.

6.4 UsuarioService.java

UsuarioService mantiene las cuentas en memoria. Incluye una cuenta administrativa inicial y métodos para buscar usuarios, comprobar credenciales, validar los campos del registro y crear una cuenta nueva. Las reglas de registro revisan que el nombre tenga una longitud mínima,

que el nombre de usuario cumpla un formato permitido, que no exista previamente, que el correo tenga una estructura válida, que la contraseña alcance la longitud requerida y que ambas contraseñas coincidan.

6.5 Servlets

ProductoServlet coordina las operaciones del inventario y contempla acciones como listar, catálogo, detalle, buscar, nuevo, guardar, editar, actualizar y eliminar. ReporteServlet prepara los datos del listado de disponibles o del resumen. InicioServlet obtiene la fecha, el total de productos y los artículos destacados. LoginServlet autentica a la persona y destruye la sesión al cerrar sesión. RegistroServlet valida la información y crea una cuenta con rol de usuario.

6.6 AuthFilter.java

AuthFilter es un filtro que intercepta las solicitudes web. Permite el acceso público a las páginas de inicio de sesión y registro, así como a recursos estáticos como CSS, JavaScript e imágenes. Para las demás rutas comprueba si existe una sesión con un usuario autenticado; si no la encuentra, redirige a la página de inicio de sesión. Este filtro establece una primera barrera de acceso, complementada por la comprobación de rol que realiza ProductoServlet en las acciones administrativas.

6.7 pom.xml y web.xml

pom.xml describe el proyecto Maven y las dependencias necesarias para compilarlo y empaquetarlo. web.xml se encuentra dentro de WEB-INF y forma parte de la configuración web; además, el proyecto utiliza anotaciones como @WebServlet y @WebFilter para declarar rutas y filtros. Por ello, la configuración de los controladores no depende exclusivamente de web.xml.

7. Explicación de los módulos funcionales

7.1 Módulo Inicio

La página principal presenta el nombre CompuTech, una descripción del sistema, la fecha actual, el usuario conectado, el total de productos y una selección de productos destacados. También ofrece accesos rápidos que cambian según el rol. El administrador ve accesos para registrar productos y revisar el inventario; el usuario común encuentra accesos al catálogo y a las consultas.

7.2 Módulo de productos

Es el componente central de la aplicación. El formulario de registro solicita código, categoría, nombre, marca, estado, precio, stock y descripción. El código identifica de manera única al

producto y se presenta como solo lectura al editarlo. Los campos de categoría y estado se alimentan de las listas definidas en ProductoService, lo que reduce la variación en los valores registrados.

La opción Listar productos muestra una tabla con código, producto, categoría, precio, stock, estado y acciones. Para el administrador se muestran las opciones Editar y Eliminar; para un usuario común se presenta la opción de consultar el detalle. La eliminación requiere confirmación mediante un cuadro de diálogo antes de enviar el formulario.

La búsqueda permite localizar productos por código, nombre o categoría. La vista de catálogo está orientada a la consulta de los artículos, mientras que la vista de detalle presenta la información de un producto seleccionado. La edición recupera los datos actuales y los carga en el formulario para que se puedan modificar y guardar.

7.3 Módulo de reportes

El reporte de productos disponibles muestra únicamente los artículos que tienen stock mayor que cero. El resumen del inventario presenta el total de productos, los disponibles, los agotados, el stock acumulado y el valor total del inventario. También presenta una comparación visual del valor de inventario por producto. Estos indicadores ayudan a identificar artículos sin unidades y a comprender la composición del inventario.

7.4 Módulo Información

La página informacion.jsp expone la descripción académica y técnica del proyecto: universidad, facultad, escuela profesional, asignatura, docente, actividad y versión. Además, explica la función de las capas Modelo, Vista y Controlador y diferencia los permisos del administrador y del usuario.

7.5 Registro e inicio de sesión

La página de inicio de sesión solicita nombre de usuario y contraseña. LoginServlet consulta UsuarioService y, si las credenciales son válidas, guarda el objeto de usuario en la sesión y redirige a InicioServlet. Si son incorrectas, muestra un mensaje de error. El registro solicita nombre, usuario, correo y contraseña repetida; si la validación termina sin errores, crea la cuenta y permite ingresar al sistema. Las cuentas nuevas reciben el rol USUARIO, no el rol administrador.

7.6 Cierre de sesión

La opción Cerrar sesión utiliza la ruta /logout. LoginServlet invalida la sesión existente y redirige al formulario de acceso con un aviso. Al invalidar la sesión, se elimina el estado de autenticación guardado para esa sesión.

8. Flujo de funcionamiento

El funcionamiento habitual empieza cuando una persona abre la aplicación. Si no ha iniciado sesión, AuthFilter la dirige al acceso. Tras autenticarse, el usuario llega a la página principal. Desde el menú puede consultar el catálogo, buscar productos y revisar los reportes permitidos por su rol. El administrador, además, puede entrar al formulario de registro, guardar nuevos artículos, editar información o eliminar registros.

En las operaciones de escritura, el formulario transmite sus valores al Servlet por medio de una solicitud POST. El controlador procesa los datos y utiliza el servicio para realizar los cambios. En las consultas, el controlador solicita al servicio una lista o un producto y envía los resultados a la página JSP. Finalmente, la vista presenta la información en tablas, tarjetas, formularios o mensajes de confirmación.

9. Validaciones y control de acceso

El formulario de producto incorpora validación con JavaScript y el servicio/controlador debe aplicar las reglas esenciales antes de modificar la colección. Los requisitos del enunciado incluyen código obligatorio y no duplicado, nombre obligatorio, precio mayor que cero, stock no negativo y categoría obligatoria. El formulario usa campos numéricos para precio y stock, listas para categoría y estado y límites de longitud para el código.

Para las cuentas, UsuarioService comprueba el formato del nombre de usuario, la duplicidad, el correo, la longitud mínima de contraseña y la coincidencia entre las dos contraseñas. AuthFilter protege las rutas internas y ProductoServlet restringe las acciones de creación, edición, actualización y eliminación al administrador. El control de acceso debe mantenerse en el servidor, porque ocultar botones en la vista no sería suficiente para proteger una operación.

Observación de seguridad: en el código revisado, las contraseñas se comparan directamente y se mantienen en memoria como texto. Esto puede servir para una práctica académica, pero no es apropiado para un sistema real. Una versión de producción debería almacenar contraseñas mediante un algoritmo de hash seguro, aplicar controles adicionales de sesión y proteger el tráfico con HTTPS.

10. Almacenamiento y gestión de datos

La actividad establece expresamente que no se debe utilizar una base de datos. Por ello, ProductoService guarda los objetos Producto en un ArrayList y UsuarioService mantiene los usuarios en otra colección. La instancia compartida de cada servicio permite que las distintas solicitudes trabajen con los mismos datos durante la ejecución del servidor.

La ventaja de este enfoque es la simplicidad: no se necesita configurar un motor de base de datos, crear tablas ni establecer conexiones JDBC. Sin embargo, el almacenamiento es temporal. Si la aplicación se reinicia o se vuelve a desplegar, los productos iniciales se cargan de nuevo y las cuentas registradas durante la ejecución pueden perderse. Por esa razón, el sistema sirve para demostrar operaciones CRUD y MVC, pero no proporciona persistencia duradera.

11. Diseño e interfaz de usuario

La interfaz utiliza una hoja de estilos centralizada en css/estilos.css y recursos gráficos ubicados en la carpeta img. Las páginas comparten encabezado, menú y pie de página mediante inclusiones JSP. Esta reutilización reduce la repetición del código y permite mantener elementos visuales comunes en las distintas secciones.

El diseño combina tablas para el inventario, formularios para registrar y editar, tarjetas para productos destacados, indicadores para los reportes y mensajes para informar errores o confirmar acciones. JavaScript apoya algunas interacciones del navegador, mientras que JSP muestra los datos que prepara el servidor. Las imágenes de productos se buscan primero por código y cuentan con una ruta alternativa basada en el nombre de imagen asociado al objeto.

12. Pruebas funcionales recomendadas

Prueba	Resultado esperado
Inicio de sesión válido	Ingresar con una cuenta registrada y verificar que se abra la página principal.
Credenciales incorrectas	Introducir una contraseña incorrecta y comprobar que aparezca un mensaje sin crear una sesión autenticada.
Registro de usuario	Crear una cuenta con datos válidos y verificar que se asigne el rol de usuario común.
Código duplicado	Intentar registrar un producto con un código existente; el sistema debe rechazarlo.
Precio inválido	Probar precio igual a cero o negativo y confirmar que la validación impida guardar.
Stock inválido	Probar un stock negativo y confirmar que no se acepte.
Búsqueda	Buscar por código, nombre y categoría y comprobar que los resultados correspondan al criterio.

Edición	Modificar el nombre o el precio de un producto y revisar que el listado muestre el cambio.
Eliminación	Cancelar la confirmación y comprobar que el registro permanezca; confirmar la eliminación y verificar que desaparezca.
Permisos	Entrar como usuario común e intentar acceder a una ruta administrativa; el servidor debe denegar la operación.
Reporte de disponibles	Comprobar que no aparezcan productos con stock cero.
Cierre de sesión	Cerrar sesión e intentar entrar directamente a una ruta privada; debe solicitar autenticación.

13. Limitaciones y mejoras propuestas

- Persistencia: incorporar una base de datos sería una mejora para una versión posterior, aunque no corresponde al requisito actual de trabajar sin base de datos.
- Seguridad de contraseñas: reemplazar el almacenamiento directo por hash seguro y revisar las políticas de sesión.
- Validación del servidor: garantizar que todas las reglas críticas se apliquen en Java, incluso si el usuario desactiva JavaScript o manipula la solicitud.
- Control de concurrencia: definir medidas para evitar inconsistencias si varios usuarios modifican el inventario al mismo tiempo.
- Auditoría: registrar quién realiza cada modificación y cuándo, si el sistema evoluciona hacia un uso real.
- Pruebas automatizadas: crear pruebas unitarias para los servicios y pruebas de integración para los Servlets.
- Accesibilidad y experiencia de usuario: revisar etiquetas de formularios, navegación con teclado, contraste y mensajes comprensibles.
- Configuración de despliegue: verificar compatibilidad entre la versión de Java, las dependencias Jakarta Servlet/JSP y Tomcat 11.

14. Conclusiones

El proyecto CompuTech desarrolla una aplicación web de gestión de productos que permite demostrar operaciones de registro, consulta, búsqueda, actualización y eliminación en un inventario de artículos informáticos.

La arquitectura MVC se refleja en la separación entre las clases del modelo, los servicios que concentran la lógica de negocio, los Servlets que coordinan las solicitudes y las páginas JSP que presentan los resultados. Esta división mejora la organización y facilita el mantenimiento del sistema.

El sistema incorpora perfiles de administrador y usuario, autenticación mediante sesión, protección general de rutas con un filtro y restricciones específicas para las acciones administrativas.

Los reportes de productos disponibles y resumen del inventario convierten los datos almacenados en indicadores útiles para revisar existencias y valor estimado.

El almacenamiento en ArrayList cumple el requisito académico de no utilizar base de datos, pero no garantiza persistencia después de reiniciar la aplicación. También se identifican oportunidades de mejora en seguridad, validación y pruebas.

15. Recomendaciones

- Ejecutar la aplicación en el entorno indicado y comprobar cada módulo mediante casos de prueba documentados.
- Mantener la separación MVC: evitar colocar lógica de negocio compleja directamente en las páginas JSP.
- Revisar que las validaciones críticas se realicen en el servidor y no dependan exclusivamente de JavaScript.
- No utilizar las credenciales demostrativas ni el manejo actual de contraseñas en un sistema real; implementar prácticas de seguridad adecuadas antes de cualquier despliegue público.
- Documentar las acciones principales, la estructura de carpetas y las instrucciones de instalación para facilitar la evaluación y el mantenimiento.